204252 Data Structures and Analysis

Chapter 8: Hashing (Part 2)

Thapanapong Rukkanchanunt and Jakarin Chawachat

August 17, 2026

Computer Science Department

Outline

Separate Chaining

Open Addressing: Linear Probing

Open Addressing: Quadratic Probing

Open Addressing: Double Hashing

Rehashing

Complexity & Summary

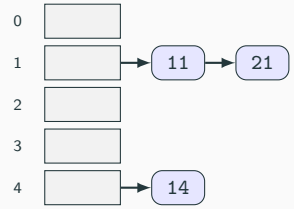
Separate Chaining

Separate Chaining Concept

The first collision resolution strategy, **Separate Chaining**, keeps a linked list of all elements that hash to the same value.

Operations:

- **Search**: Hash the key to find list index, then traverse the list.
- **Insert**: Check list for duplicate. If new, insert at the **head/front** of the list (convenient and supports temporal locality).
- **Delete**: Find the node in the list and erase it.



Separate Chaining Table

Load Factor λ

Defined as $\lambda = N/\text{TableSize}$. For separate chaining, $\lambda \approx 1$ is ideal. Average list length is λ.

Separate Chaining: C++ Class Interface

```
1  template <typename HashedObj>
2  class HashTable {
3  public:
4      explicit HashTable( int size = 101 ) : theLists( nextPrime( size ) ),
           currentSize{ 0 } {}
5      bool contains( const HashedObj & x ) const;
6      void makeEmpty();
7      bool insert( const HashedObj & x );
8      bool remove( const HashedObj & x );
9  private:
10     vector<list<HashedObj>> theLists; // Array of linked lists
11     int currentSize;
12     void rehash();
13     size_t myhash( const HashedObj & x ) const {
14         static hash<HashedObj> hf;
15         return hf( x ) % theLists.size();
16     }
17 };
```

Separate Chaining: Implementation

```
1  template <typename HashedObj>
2  bool HashTable<HashedObj>::contains( const HashedObj & x ) const {
3      auto & whichList = theLists[ myhash( x ) ];
4      return find( begin( whichList ), end( whichList ), x ) != end( whichList )
           ;
5  }
6
7  template <typename HashedObj>
8  bool HashTable<HashedObj>::insert( const HashedObj & x ) {
9      auto & whichList = theLists[ myhash( x ) ];
10     if ( find( begin( whichList ), end( whichList ), x ) != end( whichList ) )
11         return false; // Duplicate
12     whichList.push_back( x );
13     if ( ++currentSize > theLists.size() )
14         rehash(); // Load factor > 1
15     return true;
16 }
```

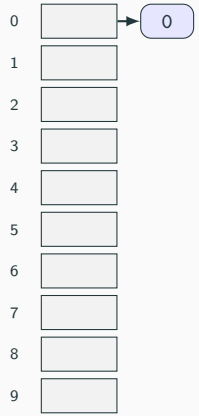
Separate Chaining: Implementation

```
1
2  template <typename HashedObj>
3  bool HashTable<HashedObj>::remove( const HashedObj & x ) {
4      auto & whichList = theLists[ myhash( x ) ];
5      auto itr = find( begin( whichList ), end( whichList ), x );
6      if ( itr == end( whichList ) ) return false; // Not found
7      whichList.erase( itr );
8      --currentSize;
9      return true;
10 }
```

Separate Chaining Insertion Trace

Let's trace inserting perfect squares: `0, 1, 4, 9, 16, 25, 36, 49, 64, 81` mod 10.

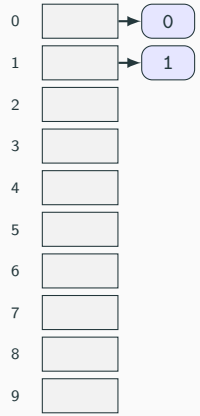
Insert 0: $h(0) = 0$. Add 0 to list 0.



Separate Chaining Insertion Trace

Let's trace inserting perfect squares: `0, 1, 4, 9, 16, 25, 36, 49, 64, 81` mod 10.

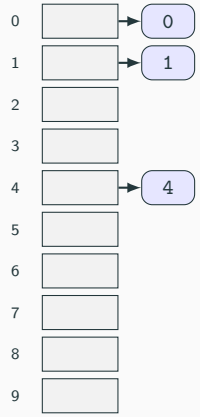
Insert 1: $h(1) = 1$. Add 1 to list 1.



Separate Chaining Insertion Trace

Let's trace inserting perfect squares: `0, 1, 4, 9, 16, 25, 36, 49, 64, 81` mod 10.

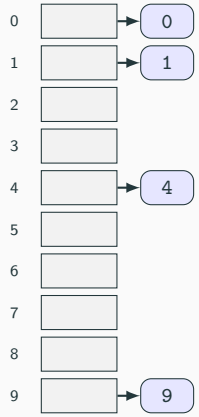
Insert 4: $h(4) = 4$. Add 4 to list 4.



Separate Chaining Insertion Trace

Let's trace inserting perfect squares: `0, 1, 4, 9, 16, 25, 36, 49, 64, 81` mod 10.

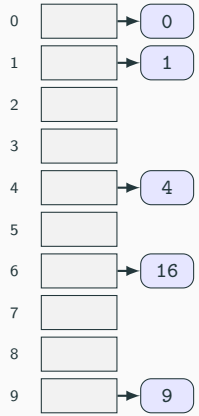
Insert 9: $h(9) = 9$. Add 9 to list 9.



Separate Chaining Insertion Trace

Let's trace inserting perfect squares: `0, 1, 4, 9, 16, 25, 36, 49, 64, 81` mod 10.

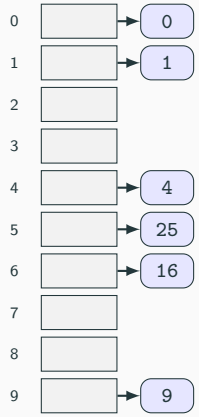
Insert 16: $h(16) = 6$. Add 16 to list 6.



Separate Chaining Insertion Trace

Let's trace inserting perfect squares: `0, 1, 4, 9, 16, 25, 36, 49, 64, 81` mod 10.

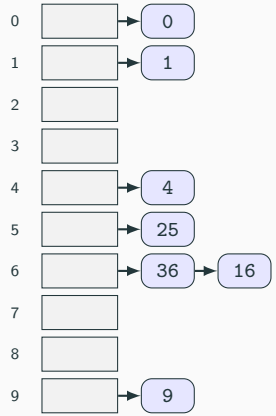
Insert 25: $h(25) = 5$. Add 25 to list 5.



Separate Chaining Insertion Trace

Let's trace inserting perfect squares: `0, 1, 4, 9, 16, 25, 36, 49, 64, 81` mod 10.

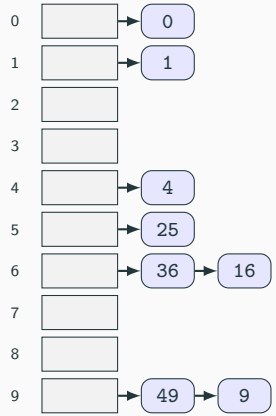
Insert 36: $h(36) = 6$. Collision! Insert 36 at head of list 6.



Separate Chaining Insertion Trace

Let's trace inserting perfect squares: `0, 1, 4, 9, 16, 25, 36, 49, 64, 81` mod 10.

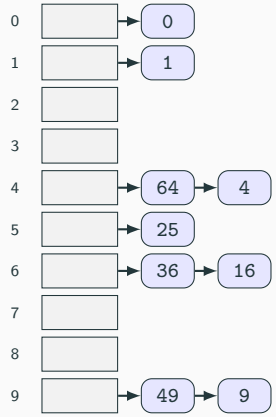
Insert 49: $h(49) = 9$. Collision! Insert 49 at head of list 9.



Separate Chaining Insertion Trace

Let's trace inserting perfect squares: `0, 1, 4, 9, 16, 25, 36, 49, 64, 81` mod 10.

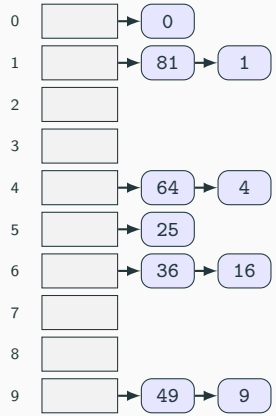
Insert 64: $h(64) = 4$. Collision! Insert 64 at head of list 4.



Separate Chaining Insertion Trace

Let's trace inserting perfect squares: `0, 1, 4, 9, 16, 25, 36, 49, 64, 81` mod 10.

Insert 81: $h(81) = 1$. Collision! Insert 81 at head of list 1.



Separate Chaining: Search and Erase Trace

Given the final table from the previous slide:

1. Search for 36 (Successful)

- Calculate hash: $h(36) = 36 \bmod 10 = 6$.
- Traverse list at index 6: First node contains **36**.
- Element found! Return true (requires 1 comparison).

Separate Chaining: Search and Erase Trace

Given the final table from the previous slide:

2. Search for 56 (Unsuccessful)

- Calculate hash: $h(56) = 56 \bmod 10 = 6$.
- Traverse list at index 6:
  - Compare 56 with 36 (mismatch, go next).
  - Compare 56 with 16 (mismatch, reached end of list).
- Element not found! Return false (requires 2 comparisons).

Separate Chaining: Search and Erase Trace

Given the final table from the previous slide:

3. Remove 16

- Calculate hash: $h(16) = 16 \bmod 10 = 6$.
- Traverse list at index 6:
  - First node: 36 (mismatch).
  - Second node: 16 (match!).
- Delete 16 and update links: index 6 now points to [36].

Open Addressing: Linear Probing

Open Addressing & Probing

For open addressing, all elements are stored directly inside the array. No linked lists!

Probing Sequence

To resolve collisions, we try alternative cells $h_0(x), h_1(x), h_2(x), \ldots$ until an empty cell is found:

$$h_i(x) = (h(x) + f(i)) \bmod \text{TableSize}$$

where $f(i)$ is the collision resolution function, and $f(0) = 0$.

Design Limitations

- A larger table size is required compared to separate chaining.
- The load factor λ must be kept below 0.5 ($\lambda < 0.5$).
- **Deletion**: Cannot perform standard deletions because it breaks search paths. Must use **Lazy Deletion**

Linear Probing

In **Linear Probing**, the collision resolution function is linear:

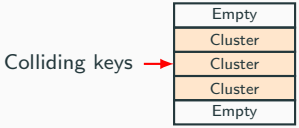
$$f(i) = i$$

This corresponds to checking subsequent cells sequentially with circular wraparound.

Primary Clustering

- Linear probing leads to blocks of occupied cells forming (clusters).
- Any key hashing into a cluster must probe through the entire cluster, making the cluster even larger.
- Insertion/unsuccessful search complexity increases rapidly as $\lambda \to 1$:

$$\text{Average probes} \approx \frac{1}{2}\left(1 + \frac{1}{(1-\lambda)^2}\right)$$



Linear Probing Trace (One slide per step)

Let's trace inserting keys: `89, 18, 49, 58, 69` into a table of size 10 ($h(k) = k \bmod 10$).

Step 1: Insert 89

- $h(89) = 9$.
- Slot 9 is `EMPTY`.
- Place 89 at index 9.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	Empty
9	89

Linear Probing Trace (One slide per step)

Let's trace inserting keys: `89`, `18`, `49`, `58`, `69` into a table of size 10 ($h(k) = k \bmod 10$).

Step 2: Insert 18

- $h(18) = 8$.
- Slot 8 is `EMPTY`.
- Place 18 at index 8.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	18
9	89

Linear Probing Trace (One slide per step)

Let's trace inserting keys: `89`, `18`, `49`, `58`, `69` into a table of size 10 ($h(k) = k \bmod 10$).

Step 3: Insert 49

- $h(49) = 9$. Collision!
- Probe $(9 + 1) \bmod 10 = 0$.
- Slot 0 is `EMPTY`.
- Place 49 at index 0.

0	49
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	18
9	89

Linear Probing Trace (One slide per step)

Let's trace inserting keys: `89`, `18`, `49`, `58`, `69` into a table of size 10 ($h(k) = k \bmod 10$).

Step 4: Insert 58

- $h(58) = 8$. Collision!
- Probe 9 (collides with 89).
- Probe 0 (collides with 49).
- Probe 1 (empty!). Place 58 at index 1.

0	49
1	58
2	Empty
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	18
9	89

Linear Probing Trace (One slide per step)

Let's trace inserting keys: `89`, `18`, `49`, `58`, `69` into a table of size 10 ($h(k) = k \bmod 10$).

Step 5: Insert 69

- $h(69) = 9$. Collision!
- Probe 0 (collides with 49).
- Probe 1 (collides with 58).
- Probe 2 (empty!). Place 69 at index 2.

0	49
1	58
2	69
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	18
9	89

Open Addressing: Quadratic Probing

Quadratic Probing

In **Quadratic Probing**, the collision resolution function is quadratic:

$$f(i) = i^2$$

This probes cells at offset distances $+1, +4, +9, +16, \ldots$ from the initial hash location.

- Eliminates primary clustering.
- **Secondary Clustering**: Elements that hash to the same position still probe the same sequence (causes a minor cost of < 0.5 extra probes).
- **Probing Limit**: No guarantee of finding an empty cell if table is $> 50\%$ full.

Quadratic Probing Trace (One slide per step)

Let's trace inserting: `89, 18, 49, 58, 69` into a table of size 10 ($h(k) = k \bmod 10, f(i) = i^2$).

Step 1: Insert 89

- $h(89) = 9$ (empty).
- Place 89 at index 9.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	Empty
9	89

Quadratic Probing Trace (One slide per step)

Let's trace inserting: `89, 18, 49, 58, 69` into a table of size 10 ($h(k) = k \bmod 10, f(i) = i^2$).

Step 2: Insert 18

- $h(18) = 8$ (empty).
- Place 18 at index 8.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	18
9	89

Quadratic Probing Trace (One slide per step)

Let's trace inserting: `89`, `18`, `49`, `58`, `69` into a table of size 10 ($h(k) = k \bmod 10, f(i) = i^2$).

Step 3: Insert 49

- $h(49) = 9$. Collision!
- Try $(9 + 1^2) \bmod 10 = 0$ (empty!).
- Place 49 at index 0.

0	49
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	18
9	89

Quadratic Probing Trace (One slide per step)

Let's trace inserting: `89`, `18`, `49`, `58`, `69` into a table of size 10 ($h(k) = k \bmod 10, f(i) = i^2$).

Step 4: Insert 58

- $h(58) = 8$. Collision!
- Try $(8 + 1^2) \bmod 10 = 9$ (collides).
- Try $(8 + 2^2) \bmod 10 = 2$ (empty!).
- Place 58 at index 2.

0	49
1	Empty
2	58
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	18
9	89

Quadratic Probing Trace (One slide per step)

Let's trace inserting: `89`, `18`, `49`, `58`, `69` into a table of size 10 ($h(k) = k \bmod 10, f(i) = i^2$).

Step 5: Insert 69

- $h(69) = 9$. Collision!
- Try $(9 + 1^2) \bmod 10 = 0$ (collides).
- Try $(9 + 2^2) \bmod 10 = 3$ (empty!).
- Place 69 at index 3.

0	49
1	Empty
2	58
3	69
4	Empty
5	Empty
6	Empty
7	Empty
8	18
9	89

Theorem 5.1 & Quadratic Probing Guarantee

Theorem 5.1

If quadratic probing is used, and the table size M is prime, then a new element can always be inserted if the table is at least half empty.

Proof: Let M be an odd prime > 3. The first $\lceil M/2 \rceil$ probe locations (including initial location $h(x)$) are:

$$h(x) + i^2 \bmod M \quad \text{and} \quad h(x) + j^2 \bmod M \quad (0 \leq i, j \leq \lfloor M/2 \rfloor)$$

Suppose for contradiction that two locations are the same, but $i \neq j$:

$$h(x) + i^2 \equiv h(x) + j^2 \pmod{M}$$
$$i^2 - j^2 \equiv 0 \pmod{M}$$
$$(i - j)(i + j) \equiv 0 \pmod{M}$$

Since M is prime, either $(i - j) \equiv 0 \pmod{M}$ or $(i + j) \equiv 0 \pmod{M}$.

- $(i - j) \equiv 0 \pmod{M}$ is impossible since $i \neq j$ and $0 \leq i, j \leq \lfloor M/2 \rfloor$.
- $(i + j) \equiv 0 \pmod{M}$ is impossible since $i + j > 0$ and $i + j \leq 2\lfloor M/2 \rfloor < M$.

Thus, all first $\lceil M/2 \rceil$ locations are distinct.

C++ Implementation of Probing HashTable

```
1  template <typename HashedObj>
2  class HashTable {
3  public:
4      explicit HashTable( int size = 101 ) : array( nextPrime( size ) ) {
           makeEmpty(); }
5      bool contains( const HashedObj & x ) const { return isActive( findPos( x )
           ); }
6      bool insert( const HashedObj & x );
7      bool remove( const HashedObj & x );
8      enum EntryType { ACTIVE, EMPTY, DELETED };
```

C++ Implementation of Probing HashTable

```
private:
    struct HashEntry {
        HashedObj element;
        EntryType info;
        HashEntry( const HashedObj & e = HashedObj{ }, EntryType i = EMPTY ) :
             element{ e }, info{ i } {}
    };
    vector<HashEntry> array;
    int currentSize;
    bool isActive( int currentPos ) const { return array[ currentPos ].info ==
         ACTIVE; }
    int findPos( const HashedObj & x ) const;
    void rehash();
};
```

Probing implementation: findPos and insert

```
1  template <typename HashedObj>
2  int HashTable<HashedObj>::findPos( const HashedObj & x ) const {
3      int offset = 1;
4      int currentPos = myhash( x );
5      while ( array[ currentPos ].info != EMPTY && array[ currentPos ].element
           != x ) {
6          currentPos += offset; // F(i) = F(i-1) + 2i - 1
7          offset += 2;
8          if ( currentPos >= array.size() )
9              currentPos -= array.size();
10     }
11     return currentPos;
12 }
```

Probing implementation: findPos and insert

```
1  template <typename HashedObj>
2  bool HashTable<HashedObj>::insert( const HashedObj & x ) {
3      int currentPos = findPos( x );
4      if ( isActive( currentPos ) ) return false; // Duplicate
5      array[ currentPos ].element = x;
6      array[ currentPos ].info = ACTIVE;
7      if ( ++currentSize > array.size() / 2 )
8          rehash(); // Load factor > 0.5
9      return true;
10 }
```

Open Addressing: Double Hashing

Double Hashing

In **Double Hashing**, the collision resolution function is:

$$f(i) = i \cdot h_2(x)$$

where $h_2(x)$ is a secondary hash function used to calculate the probe step size.

Design of $h_2(x)$

- $h_2(x)$ must **never** evaluate to 0.
- TableSize M must be prime to ensure that all cells can be probed.
- A common choice is:

  $$h_2(x) = R - (x \bmod R)$$

  where R is a prime number smaller than the table size M.
- Practically, double hashing is theoretically interesting as it matches random probing, but quadratic probing is simpler and doesn't require a second hash function.

Double Hashing Trace (One slide per step)

Let's trace inserting: `89, 18, 49, 58, 69` into a table of size 10.

- $h_1(x) = x \bmod 10$, and $h_2(x) = 7 - (x \bmod 7)$ (where $R = 7$).

Step 1: Insert 89

- $h_1(89) = 9$.
- Slot 9 is empty.
- Place 89 at index 9.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	Empty
9	89

Double Hashing Trace (One slide per step)

Let's trace inserting: `89, 18, 49, 58, 69` into a table of size 10.

- $h_1(x) = x \bmod 10$, and $h_2(x) = 7 - (x \bmod 7)$ (where $R = 7$).

Step 2: Insert 18

- $h_1(18) = 8$.
- Slot 8 is empty.
- Place 18 at index 8.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	Empty
7	Empty
8	18
9	89

Double Hashing Trace (One slide per step)

Let's trace inserting: `89, 18, 49, 58, 69` into a table of size 10.

- $h_1(x) = x \bmod 10$, and $h_2(x) = 7 - (x \bmod 7)$ (where $R = 7$).

Step 3: Insert 49

- $h_1(49) = 9$. Collision!
- Compute step size:
  $h_2(49) = 7 - (49 \bmod 7) = 7$.
- Try $(9 + 7) \bmod 10 = 6$ (empty!).
- Place 49 at index 6.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	49
7	Empty
8	18
9	89

Double Hashing Trace (One slide per step)

Let's trace inserting: `89, 18, 49, 58, 69` into a table of size 10.

- $h_1(x) = x \bmod 10$, and $h_2(x) = 7 - (x \bmod 7)$ (where $R = 7$).

Step 4: Insert 58

- $h_1(58) = 8$. Collision!
- Compute step size:
  $h_2(58) = 7 - (58 \bmod 7) = 5$.
- Try $(8 + 5) \bmod 10 = 3$ (empty!).
- Place 58 at index 3.

0	Empty
1	Empty
2	Empty
3	58
4	Empty
5	Empty
6	49
7	Empty
8	18
9	89

Double Hashing Trace (One slide per step)

Let's trace inserting: `89, 18, 49, 58, 69` into a table of size 10.

- $h_1(x) = x \bmod 10$, and $h_2(x) = 7 - (x \bmod 7)$ (where $R = 7$).

Step 5: Insert 69

- $h_1(69) = 9$. Collision!
- Compute step size:
  $h_2(69) = 7 - (69 \bmod 7) = 1$.
- Try $(9 + 1) \bmod 10 = 0$ (empty!).
- Place 69 at index 0.

0	69
1	Empty
2	Empty
3	58
4	Empty
5	Empty
6	49
7	Empty
8	18
9	89

Rehashing

Rehashing

When the load factor becomes too large, operations start degrading.

The Rehash Operation

- Triggered when λ crosses a threshold (e.g. $\lambda > 0.5$ for probing, $\lambda > 1.0$ for chaining).
- Create a new table that is roughly **twice as large** (its size must be the next prime number).
- Scan the old table and re-insert all active elements into the new table using a **new hash function** based on the new size.
- Complexity: $\mathcal{O}(N)$ operation, but since it happens infrequently, the amortized cost per insertion remains $\mathcal{O}(1)$.

Rehashing Trace: Setup

Suppose we insert `13, 15, 24, 6` into a linear probing table of size 7 ($h(x) = x \bmod 7$).

Initial Table Setup:

- $h(13) = 6 \rightarrow$ slot 6.
- $h(15) = 1 \rightarrow$ slot 1.
- $h(24) = 3 \rightarrow$ slot 3.
- $h(6) = 6 \rightarrow$ collision at 6, probe $(6 + 1) \bmod 7 = 0 \rightarrow$ slot 0.

0	6
1	15
2	Empty
3	24
4	Empty
5	Empty
6	13

Rehashing Trace: Trigger

Now, insert key 23.

Inserting 23:

- $h(23) = 23 \bmod 7 = 2$.
- Slot 2 is empty. Place 23 at index 2.

Rehash Triggered!
The table contains 5 elements. The size is 7.

$$\lambda = 5/7 \approx 71.4\% > 50\%$$

We must double the size: $7 \times 2 = 14$. The first prime > 14 is **17**.

New Hash Function:

$$h'(x) = x \bmod 17$$

0	6
1	15
2	23
3	24
4	Empty
5	Empty
6	13

Rehashing Trace: Execution

Scan the old table and re-insert elements: `6, 15, 23, 24, 13` into the new table of size 17 ($h'(x) = x \bmod 17$).

Rehash 6

- $h'(6) = 6 \bmod 17 = 6$.
- Place 6 at index 6.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	6
7	Empty
8	Empty
9	Empty
10	Empty
11	Empty
12	Empty
13	Empty
14	Empty
15	Empty
16	Empty

Rehashing Trace: Execution

Scan the old table and re-insert elements: `6, 15, 23, 24, 13` into the new table of size 17 ($h'(x) = x \bmod 17$).

Rehash 15

- $h'(15) = 15 \bmod 17 = 15$.
- Place 15 at index 15.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	6
7	Empty
8	Empty
9	Empty
10	Empty
11	Empty
12	Empty
13	Empty
14	Empty
15	15
16	Empty

Rehashing Trace: Execution

Scan the old table and re-insert elements: `6, 15, 23, 24, 13` into the new table of size 17 ($h'(x) = x \bmod 17$).

Rehash 23

- $h'(23) = 6$. Collision!
- Probe $(6 + 1) \bmod 17 = 7$ (empty!).
- Place 23 at index 7.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	6
7	23
8	Empty
9	Empty
10	Empty
11	Empty
12	Empty
13	Empty
14	Empty
15	15
16	Empty

Rehashing Trace: Execution

Scan the old table and re-insert elements: `6, 15, 23, 24, 13` into the new table of size 17 ($h'(x) = x \bmod 17$).

Rehash 24

- $h'(24) = 24 \bmod 17 = 7$. Collision!
- Probe $(7 + 1) \bmod 17 = 8$ (empty!).
- Place 24 at index 8.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	6
7	23
8	24
9	Empty
10	Empty
11	Empty
12	Empty
13	Empty
14	Empty
15	15
16	Empty

Rehashing Trace: Execution

Scan the old table and re-insert elements: `6, 15, 23, 24, 13` into the new table of size 17 ($h'(x) = x \bmod 17$).

Rehash 13

- $h'(13) = 13 \bmod 17 = 13$.
- Place 13 at index 13.

0	Empty
1	Empty
2	Empty
3	Empty
4	Empty
5	Empty
6	6
7	23
8	24
9	Empty
10	Empty
11	Empty
12	Empty
13	13
14	Empty
15	15
16	Empty

Complexity & Summary

Average vs Worst-Case Complexity

Operation	Average Case	Worst Case
Separate Chaining Search (Successful)	$\mathcal{O}(1 + \lambda/2)$	$\mathcal{O}(N)$ (all keys hash to 1 list)
Separate Chaining Search (Unsuccessful)	$\mathcal{O}(1 + \lambda)$	$\mathcal{O}(N)$
Separate Chaining Insertion	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Linear Probing Search (Successful)	$\mathcal{O}(1 + 1/(1 - \lambda))$	$\mathcal{O}(N)$ (must scan whole table)
Linear Probing Search (Unsuccessful)	$\mathcal{O}(1 + 1/(1 - \lambda)^2)$	$\mathcal{O}(N)$
Quadratic Probing Search (Successful)	$\mathcal{O}(1 + \ln(1/(1 - \lambda))/\lambda)$	$\mathcal{O}(N)$
Double Hashing Search (Successful)	$\mathcal{O}(1 + \ln(1/(1 - \lambda))/\lambda)$	$\mathcal{O}(N)$

Table 1: Hashing Complexity Analysis

Summary

Hashing provides $\mathcal{O}(1)$ search in the average case. If λ is managed properly ($\lambda \approx 1$ for chaining, $\lambda \leq 0.5$ for probing) and rehashing is performed when necessary, Hashing outperforms Trees for non-ordered data.